# OpenStack CLI Field Guide

# OpenStack CLI Field Guide

*House of the Cloud, Hexworth Prime. Cloud release: OpenStack 2026.1 “Gazpacho”.*

## What you are connected to

This is **not a simulator**. The Live Cloud Terminal on the OpenStack hub opens a shell wired to a real OpenStack cloud running on Hexworth infrastructure. When you type `openstack server list`, a real cloud controller answers.

Your access is a **read-only demo project**. You can list, describe, and inspect anything in it; you cannot create, modify, or delete. That is enforced by your account’s *role*, not by hiding buttons. Try a write and you will get a real `403 Forbidden` from the cloud, which is itself worth seeing once.

**How to get in:** OpenStack hub, then *Live Cloud Terminal*, then **Launch Sandbox**, then **Open Terminal**. Your session lasts up to 120 minutes and idles out after 15. Relaunching gives you a fresh shell; the cloud on the other side keeps its state.

**One thing to know before you start:** every command begins with `openstack`. It is one unified client for every service, compute, networking, storage, identity. That is a deliberate design choice you will appreciate the first time you use a cloud that has a different CLI per service.

---

## The commands you will actually use

Your shell already has credentials loaded (`OS_CLOUD=demo`), so you never type a password.

## Look around

| Command | What it answers |
|---|---|
| `openstack server list` | What instances exist in my project? |
| `openstack server show <name>` | Everything about one instance: state, IPs, flavor, image |
| `openstack flavor list` | What instance sizes may I choose from? |
| `openstack image list` | What operating system images can I boot? |
| `openstack network list` | What networks exist? |
| `openstack subnet list` | What IP ranges live inside those networks? |
| `openstack router list` | What connects my private network to the outside? |
| `openstack volume list` | What block-storage volumes exist? |
| `openstack security group list` | What firewall rule groups exist? |
| `openstack keypair list` | What SSH keys are registered? |
| `openstack quota show` | What am I allowed to consume? |
| `openstack --help` | The full command tree (it is large, so pipe it through `less`) |

## Make the output usable

The CLI is designed to be scripted, and these three habits are the difference between fighting it and using it:

| Command | Effect |
|---|---|
| `openstack server list -f value -c Name` | Just the names, no table borders, ideal in scripts |
| `openstack server list -f json` | JSON out, ready for `jq` |
| `openstack server show <name> -c status -c addresses` | Only the columns you asked for |
| `openstack server list --long` | Extra columns most people never discover |

Try `openstack server list -f value -c Name | wc -l`. That is the moment the CLI stops being a worse version of the web console and starts being a tool.

### Writes: what they look like when you have the role

You cannot run these in the read-only lab. They are here because they are the commands the next lab teaches, and because reading them now makes the shape of the launch chain obvious:

```
openstack keypair create mykey > mykey.pem
openstack network create demo-net
openstack subnet create demo-subnet --network demo-net --subnet-range 10.0.0.0/24
openstack security group create demo-sg
openstack security group rule create --proto tcp --dst-port 22 demo-sg
openstack server create --flavor m1.nano --image cirros --network demo-net \
    --security-group demo-sg --key-name mykey myserver
openstack floating ip create public
openstack server add floating ip myserver <floating-ip>
```

Count the dependencies in that chain: **seven objects must exist and reference each other correctly before one instance is reachable over SSH.** Every cloud makes you satisfy the same seven; only the names change. That is the whole point of learning it here.

---

## The transfer table: why this is worth your time

You are not learning OpenStack because your employer runs OpenStack. You are learning the *concepts*, which are identical across every major cloud. Learn the middle column once and the other two are vocabulary swaps.

| Concept | OpenStack | AWS | Azure |
|---|---|---|---|
| Virtual machine | Nova instance | EC2 instance | Virtual Machine |
| VM size | Flavor | Instance type | VM size |
| Boot image | Glance image | AMI | Image / Gallery image |
| Block disk | Cinder volume | EBS volume | Managed Disk |
| Object storage | Swift container | S3 bucket | Blob container |
| Private network | Neutron network | VPC | Virtual Network (VNet) |
| Subnet | Subnet | Subnet | Subnet |

| Concept | OpenStack | AWS | Azure |
|---|---|---|---|
| Public IP | Floating IP | Elastic IP | Public IP address |
| Router / gateway | Neutron router | Internet Gateway + route table | VNet gateway / routes |
| Instance firewall | Security group | Security group | Network Security Group |
| Identity + permissions | Keystone users/projects/roles | IAM users/accounts/policies | Entra ID + RBAC |
| Tenancy boundary | Project | Account | Subscription / Resource Group |
| Usage ceiling | Quota | Service quota | Subscription limit |
| Web console | Horizon | AWS Console | Azure Portal |
| Unified CLI | `openstack` | `aws` | `az` |
| Orchestration | Heat template | CloudFormation | ARM / Bicep |

**Interview-grade takeaway:** if you can explain why an instance with a public IP still refuses SSH until a security group rule allows port 22, you understand cloud networking better than the candidate who memorized console clicks. The failure is identical on all three clouds.

---

## Concepts, in the order they bite you

**Project (tenant).** Every resource belongs to a project, and your token is scoped to one. This is why `server list` shows *your* project's servers and nothing else, not a filter, a boundary. Ask for another project's resources and the answer is 403.

**Role.** Your project membership says *where* you can act; your role says *what* you may do. `reader` reads. `member` creates and deletes. Same account, same project, completely different power. When you are denied, the useful question is "which role am I missing?" not "is the system broken?"

**Flavor + image = instance.** The flavor is the hardware profile (vCPU, RAM, disk); the image is the operating system. Neither is the instance. Boot = flavor + image + network, at minimum.

**Network, subnet, router and floating IP: four objects, four jobs.** The network is an L2 domain. The subnet hands out addresses inside it. The router bridges it to the outside world. The floating IP is a public address *mapped* onto an instance's private one. Skipping the router is the classic "my instance has an IP but cannot reach anything" mistake.

**Security group.** A stateful allow-list attached to an instance. Default posture is deny. A brand-new instance answers nothing until you allow something. This is a feature, and it is the single most common reason a fresh cloud VM "does not work."

**Volume.** Storage with a life independent of the instance. Delete the instance and a properly detached volume still holds your data. This is the difference between a disk and a filesystem, and it is the concept students most often think they understand until they test it.

**Quota.** A ceiling per project. You will hit one eventually, and the error will look like a system failure rather than a policy limit. See the decoder below.

---

# Error decoder

Cloud errors are terse and easy to misread. These are the ones you will actually meet:

| What you see | What it means | What to do |
|---|---|---|
| `403 Forbidden` / `disallowed by policy` | Your role does not permit this action | Correct behavior in this lab, you are a reader. Note *which* action was denied |
| `No valid host found` | The scheduler could not place your instance, usually no host has enough free RAM/CPU for that flavor | Try a smaller flavor; check quota and capacity. Not a bug |
| Instance stuck in `BUILD` | Scheduling or image download is still in progress, or wedged | `openstack server show` and read `fault`; check the compute service is up |
| SSH times out on a floating IP | Almost always no security-group rule for port 22 | Check the group's rules before suspecting the network |

| What you see | What it means | What to do |
|---|---|---|
| `Quota exceeded for instances` | You hit the project ceiling, not a failure | Delete something you are done with, or ask for more |
| Instance `ACTIVE` but no IP | Booted with no network, or DHCP never answered | `server show` and inspect `addresses` |
| `401 Unauthorized` | Token expired or credentials wrong | Relaunch the lab to get fresh credentials |

**Habit worth building now:** when a cloud command fails, read the *whole* error. OpenStack tells you the policy rule name or the scheduler reason. Guessing wastes the information it just handed you.

---

# Official documentation

Every link below was verified live before this sheet was printed. These are the same upstream references professional operators use. Bookmark the first two.

### Start here

- OpenStack CLI reference (all commands): https://docs.openstack.org/python-openstackclient/latest/cli/index.html
- Full command index by service: https://docs.openstack.org/python-openstackclient/latest/cli/command-list.html

### Command reference, by object

| You are working with | Reference |
|---|---|
| Instances (`server ...`) | https://docs.openstack.org/python-openstackclient/latest/cli/command-objects/compute/v2/index.html#server |
| Flavors | https://docs.openstack.org/python-openstackclient/latest/cli/command-objects/compute/v2/index.html#flavor |
| SSH key pairs | https://docs.openstack.org/python-openstackclient/latest/cli/command-objects/compute/v2/index.html#keypair |

| You are working with | Reference |
| --- | --- |
| Networks | https://docs.openstack.org/python-openstackclient/latest/cli/command-objects/network/v2/index.html#network |
| Subnets | https://docs.openstack.org/python-openstackclient/latest/cli/command-objects/network/v2/index.html#subnet |
| Routers | https://docs.openstack.org/python-openstackclient/latest/cli/command-objects/network/v2/index.html#router |
| Floating IPs | https://docs.openstack.org/python-openstackclient/latest/cli/command-objects/network/v2/index.html#floating-ip |
| Security group rules | https://docs.openstack.org/python-openstackclient/latest/cli/command-objects/network/v2/index.html#security-group-rule |
| Volumes (create, list, extend) | https://docs.openstack.org/python-openstackclient/latest/cli/command-objects/volume/v2/index.html#volume |
| Quotas | https://docs.openstack.org/python-openstackclient/latest/cli/command-objects/common/index.html#quota |

Worth knowing: **attaching** a volume is not a `volume` command. It is `openstack server add volume`, documented under the *server* object, because the operation belongs to the instance. The Cinder lifecycle guide below shows attach and detach together with worked examples.

**Concept and task guides**

| Topic | Guide |
| --- | --- |
| Launch an instance, end to end | https://docs.openstack.org/nova/2026.1/user/launch-instances.html |
| Security groups explained | https://docs.openstack.org/nova/2026.1/user/security-groups.html |
| Volume attach, detach, extend | https://docs.openstack.org/cinder/2026.1/cli/cli-manage-volumes.html#attach-a-volume-to-an-instance |

| Topic | Guide |
| --- | --- |
| How the scheduler picks a host | https://docs.openstack.org/nova/2026.1/admin/scheduling.html |

On that last one, an honest note: the scheduling page explains the filters that *cause* a `No valid host found` result. There is no upstream page that walks you through fixing it. Diagnosing it is log triage plus capacity checking, which is exactly how it works in industry too.

**Per-service documentation**

Keystone (identity) https://docs.openstack.org/keystone/2026.1/ · Nova (compute) https://docs.openstack.org/nova/2026.1/ · Neutron (networking) https://docs.openstack.org/neutron/2026.1/ · Cinder (block storage) https://docs.openstack.org/cinder/2026.1/ · Glance (images) https://docs.openstack.org/glance/2026.1/ · Horizon (dashboard) https://docs.openstack.org/horizon/2026.1/

**About this release**

The cloud you are using runs **OpenStack 2026.1 “Gazpacho”**, released 1 April 2026. It is the 33rd OpenStack release, built by roughly 500 contributors from 100 organizations.

- Release page: https://releases.openstack.org/gazpacho/index.html
- What is new in this release: https://releases.openstack.org/gazpacho/highlights.html
- Project home: https://www.openstack.org/software/openstack-gazpacho/
- Documentation hub: https://docs.openstack.org/2026.1/

---

# Your session, honestly

- **120 minutes max**, 15-minute idle timeout, 2 concurrent sandboxes per person.
- The platform pool is shared (40 concurrent across all Hexworth labs); graded work has priority.
- Relaunching is free and gives a clean shell. Nothing you can do in a read-only session breaks anything, so explore aggressively.
- What you cannot do yet: create instances of your own. Per-student cloud projects with write access are the next stage of this lab; your work will then persist between sessions.